

PAC-MAN

Implementación en lenguaje C

Informe de Proyecto Final

Fundamentos de Computación I — IC213
Universidad Nacional de Misiones

Alumno: Santiago Lucas Nahuel Javier
Año: 2026

1. Introducción

El presente proyecto consiste en el desarrollo de una versión del videojuego clásico **Pac-Man**, implementada íntegramente en lenguaje **C** y orientada a la consola de Windows. El juego original fue creado por Namco en 1980 y se convirtió en uno de los íconos más reconocidos de la historia de los videojuegos.

El proyecto surge como trabajo final de la materia **Fundamentos de Computación I**, con el objetivo de aplicar de forma integrada los conceptos de estructuras de datos lineales y no lineales, diseño modular, manejo de archivos y algoritmos de búsqueda estudiados a lo largo del curso.

El juego se desarrolla en una grilla de 19×19 celdas renderizada en la terminal mediante caracteres Unicode y la API de consola de Windows. El jugador controla a Pac-Man con las teclas W/A/S/D, debe consumir todas las monedas del mapa evitando a cuatro fantasmas que se mueven con inteligencia artificial diferenciada, y puede utilizar power-ups para invertir temporalmente los roles.

Alcance y condiciones de contorno

- El juego corre exclusivamente en **Windows** (uso de windows.h, conio.h, funciones de consola).
- La interfaz es completamente **textual** (terminal), sin gráficos externos ni librerías multimedia.
- Se implementan **tres niveles de dificultad** que modifican la velocidad del ciclo de juego.
- El sistema de **highscores** persiste en el archivo highscore.txt.
- No se implementa guardado de partida en curso (save/load de estado).

2. Objetivos específicos del proyecto

- Implementar el movimiento y la lógica de juego de Pac-Man en un entorno de consola.
- Representar el mapa como una **matriz bidimensional** y construir sobre ella un **grafo de adyacencia** para el pathfinding.
- Implementar el algoritmo **BFS (Búsqueda en Anchura)** sobre el grafo utilizando una **cola dinámica con nodos enlazados** como estructura lineal obligatoria.
- Desarrollar las **cuatro personalidades clásicas** de los fantasmas (Blinky, Pinky, Inky, Clyde) con sus comportamientos originales del arcade.
- Aplicar un diseño **modular** con separación clara de responsabilidades entre archivos.
- Implementar un **sistema de puntaje** con bonus por velocidad y multiplicador según dificultad, con persistencia en archivo.
- Construir un **menú animado** con selección de dificultad y visualización de highscores.

3. Diagrama en bloques del sistema

El sistema se organiza en seis módulos principales. El flujo de control parte desde **main.c**, que actúa como orquestador central, e invoca a los demás módulos según la etapa del programa:

Módulo	Archivo(s)	Rol en el sistema
main	main.c	Punto de entrada. Orquesta el loop del juego, la lógica de colisiones, el puntaje y la pausa.
menu	menu.c / menu.h	Menú principal animado y menú de selección de nivel. Devuelve la opción elegida.
mapa	mapa.c / mapa.h	Define y almacena la grilla 19×19. Provee reinicio del mapa y renderizado visual.
personajes	personajes.c / personajes.h	Define y opera sobre Pac-Man y los Fantasmas: movimiento, sprites, colisión.
grafo	grafo.c / grafo.h	Construye el grafo de adyacencia sobre el mapa y ejecuta BFS para el pathfinding.
cola	cola.c / cola.h	Cola dinámica con lista enlazada. Usada internamente por BFS en grafo.c.

Las dependencias entre módulos siguen una jerarquía clara: **main** depende de todos; **grafo** depende de **mapa**, **personajes** y **cola**; **mapa** y **personajes** dependen mutuamente; **cola** no depende de ningún otro módulo del proyecto.

4. Descripción de la información intercambiada entre módulos

Origen → Destino	Dato intercambiado	Mecanismo
main → mapa	Llamada a reiniciarMapa() al inicio de cada partida	Función sin parámetros
main → personajes	Posición inicial de Pac-Man y fantasmas; dirección de movimiento	Punteros a structs Pacman / Fantasma
main → grafo	Structs Fantasma*, Pacman*, Grafo* para calcular el próximo movimiento	Punteros pasados a moverFantasmaInteligente()
main ← mapa	Valor de mapa[fila][col] para detectar PUNTO, POWER, CAMINO	Variable global extern int mapa[][]
grafo → mapa	Lee mapa[][] para construir aristas; consulta PARED para filtrar celdas	Variable global extern
grafo → cola	Encola/desencola IDs de nodos durante BFS	Puntero a Cola; API encolar/desencolar
grafo → personajes	Lee fila/col de Fantasma y Pacman; escribe fila/col del fantasma tras BFS	Punteros a structs
personajes → mapa	Lee mapa[fila][col] para detectar paredes; modifica PUNTO→CAMINO al comer	Variable global + función esCamino()
menu → main	OpcionMenu seleccionada (enum); nivel elegido (int 1-3)	Valor de retorno de mostrarMenu() y mostrarMenuNivel()

5. Funcionalidad de cada módulo

5.1 Módulo main (main.c)

Es el núcleo del programa. Contiene el loop principal del juego y coordina todos los demás módulos. Define la struct **ConfigNivel** que agrupa los parámetros de cada dificultad: sleepMs (velocidad del ciclo), velocidadFantasmas (cada cuántos turnos se mueven los fantasmas), multiplicadorBonus (factor sobre el bonus de tiempo) y nombre.

Función	Qué hace	Parámetros / Retorno
limpiarPantalla()	Mueve el cursor a (0,0) para sobrescribir el frame anterior sin parpadeo visible.	void → void
ocultarCursor()	Oculta el cursor parpadeante de la consola durante el juego.	void → void
contarMonedas()	Recorre mapa[][] contando celdas PUNTO y POWER restantes. Retorna 0 cuando el jugador ganó.	void → int
calcularBonusTiempo()	Devuelve (180 - segundos) × 10 si se terminó antes de 180 s, o 0 si no.	int segundos → int
mostrarVictoria()	Muestra pantalla de victoria, pide nombre con do-while (no permite campo vacío) y guarda en highscore.txt.	int, int, const char* → void
mostrarHighscore()	Lee highscore.txt y muestra los primeros 5 registros en tabla ASCII.	void → void
mostrarInstrucciones()	Imprime la pantalla estática de controles.	void → void
jugar(int nivel)	Loop principal: entrada del jugador, movimiento de fantasmas, colisiones, power-ups, pausa, renderizado y condición de victoria/derrota.	int nivel → void
main()	Inicializa la semilla aleatoria y ejecuta el loop del menú principal hasta que el usuario elija salir.	void → int

Detalles del loop de juego

- Cada iteración guarda las posiciones anteriores de Pac-Man y los fantasmas para detectar colisiones cruzadas.
- Si hay tecla disponible (`_kbhit()`), la procesa; si no, repite la última dirección de Pac-Man.
- Detecta consumo de POWER-UP: activo modo asustado durante `DURACION_ASUSTADO` turnos.
- La **pausa** congela el loop: espera `ESPACIO`, luego suma el tiempo pausado a `tiempoInicio` para no penalizar el cronómetro.
- Los fantasmas solo se mueven cada `cfg.velocidadFantasmas` turnos (1, 2 o 3 según dificultad).

5.2 Módulo mapa (mapa.c / mapa.h)

Representa el escenario como una matriz de enteros 19x19. Define cuatro valores de celda: **PARED (0)**, **CAMINO (1)**, **PUNTO (2)** y **POWER (3)**. El diseño clave es separar el mapa editable del mapa original: mapaOriginal es static const y nunca cambia; mapa[][] es la copia activa que se modifica al consumir monedas. Esto permite reiniciar la partida sin relanzar el proceso.

Función	Qué hace	Parámetros / Retorno
reiniciarMapa()	Copia mapaOriginal en mapa[][] usando memcpy(). Restaura todas las monedas y power-ups al estado inicial.	void → void
imprimirMapa()	Construye un buffer mapaVisual[19][19][8] con sprites por celda, sobrescribe con fantasmas y Pac-Man, e imprime fila a fila.	Pacman*, Fantasma[], int n → void
esCamino()	Retorna 1 si la celda (fila, col) existe dentro de los límites y no es PARED. Usada para el movimiento de ambas entidades.	int fila, int col → int

5.3 Módulo personajes (personajes.c / personajes.h)

Define las estructuras de datos y el comportamiento de los dos tipos de entidades del juego.

Struct Pacman

- **fila, col**: posición actual en el mapa.
- **vidas**: contador de vidas (inicia en 3).
- **puntaje**: puntos acumulados.
- **dir**: última dirección válida (enum Direccion: ARRIBA, ABAJO, IZQUIERDA, DERECHA, QUIETO).
- **sprite[10]**: representación visual actual.
- **tiempoInicio**: timestamp de inicio de partida (time_t de time.h).

Struct Fantasma

- **fila, col**: posición actual.
- **tipo**: enum TipoFantasma (BLINKY, PINKY, INKY, CLYDE).
- **asustado**: bandera booleana (1 = modo asustado).
- **sprite[10]**: sprite normal (emoji de color según tipo).
- **spriteAsustado[10]**: sprite alternativo cuando está asustado (emoji asustado).

Función	Qué hace	Parámetros / Retorno
iniciarPacman()	Inicializa todos los campos del struct Pacman en sus valores por defecto.	Pacman*, int fila, int col → void
moverPacman()	Calcula nueva posición según dirección. Si esCamino(), actualiza posición, consume PUNTO (+10 pts) y actualiza sprite.	Pacman*, Direccion → void

Función	Qué hace	Parámetros / Retorno
actualizarSpritePacman()	Asigna sprite direccional: C^ arriba, Cv abajo, <C izquierda, C> derecha.	Pacman* → void
iniciarFantasma()	Inicializa posición, tipo, dirección y ambos sprites del fantasma.	Fantasma*, TipoFantasma, int, int → void
moverFantasma()	Movimiento aleatorio: elige entre direcciones válidas excluyendo el sentido contrario al actual.	Fantasma* → void
asustarFantasma()	Pone asustado=1 y cambia el sprite a SPRITE_ASUSTADO en todos los fantasmas.	Fantasma[], int n → void
calmarFantasma()	Restaura asustado=0 y el sprite original según el tipo de fantasma.	Fantasma[], int n → void
colisionPacmanFantasma()	Retorna 1 si Pac-Man y el fantasma comparten la misma celda.	Pacman*, Fantasma* → int

5.4 Módulo grafo (grafo.c / grafo.h)

Es el módulo más complejo del sistema. Implementa el grafo de navegación del mapa y contiene el algoritmo BFS y las cuatro personalidades de los fantasmas.

Estructuras de datos

- **NodoAdj**: nodo de lista de adyacencia. Contiene destino (ID de celda vecina) y puntero *sig.
- **Nodo**: cabeza de lista de adyacencia para un nodo del grafo (NodoAdj *cabeza).
- **Grafo**: arreglo de MAX_NODOS = FILAS × COLS = 361 nodos, más blinkyFila y blinkyCol para el cálculo de Inky.

Cada celda no-pared se convierte en un nodo con ID = fila × COLS + col. Para cada par de celdas adyacentes (4 direcciones) que no sean paredes, se agrega una arista dirigida. El grafo resultante representa todos los caminos transitables del mapa.

Función	Qué hace	Parámetros / Retorno
celda_a_id()	Convierte coordenadas (fila, col) al ID lineal del nodo.	int, int → int
id_a_celda()	Operación inversa: descompone el ID en fila y columna.	int, int*, int* → void
construirGrafo()	Recorre todas las celdas no-pared y agrega aristas hacia sus vecinos válidos. Inicializa blinkyFila/Col.	Grafo* → void
bfsProximoPaso()	BFS desde origen hasta destino. Reconstruye el camino con el arreglo padre[] y retorna el ID del primer paso.	Grafo*, int, int → int
moverFantasmaInteligente()	Despacha según el tipo del fantasma: calcula el destino objetivo y luego ejecuta BFS para obtener el próximo paso.	Fantasma*, Pacman*, Grafo* → void

Función	Qué hace	Parámetros / Retorno
liberarGrafo()	Libera todos los NodoAdj asignados dinámicamente con malloc().	Grafo* → void

Personalidades de los fantasmas

Fantasma	Nombre	Lógica del objetivo
BLINKY (rojo)	El perseguidor	Destino = posición actual de Pac-Man. Persecución directa mediante BFS.
PINKY (violeta)	La emboscadora	Destino = 4 celdas adelante de la dirección de Pac-Man. Intenta interceptarlo antes de que llegue.
INKY (azul)	El impredecible	Ref = 2 celdas adelante de Pac-Man. Destino = Ref + (Ref - Blinky). Vector duplicado desde la posición de Blinky.
CLYDE (naranja)	El tímido	Si dist(Clyde, Pac-Man) > 8: persigue como Blinky. Si dist ≤ 8: huye a la esquina inferior-izquierda del mapa.

5.5 Módulo cola (cola.c / cola.h)

Implementa una **cola FIFO dinámica** usando una lista simplemente enlazada. Es la estructura de datos lineal usada exclusivamente por el BFS en grafo.c.

Structs NodoCola / Cola

- **NodoCola**: nodo con int valor (ID de celda) y puntero *sig al siguiente.
- **Cola**: struct con punteros frente y fin, más contador cantidad.

Función	Qué hace
iniciarCola()	Inicializa frente=NULL, fin=NULL, cantidad=0.
encolar()	Crea un NodoCola con malloc(), lo inserta al final de la lista enlazada.
desencolar()	Extrae el nodo del frente, libera su memoria con free() y retorna el valor. Retorna -1 si está vacía.
colaVacia()	Retorna 1 si frente==NULL.
liberarCola()	Llama a desencolar() hasta vaciar la cola completamente, liberando toda la memoria.

5.6 Módulo menu (menu.c / menu.h)

Maneja toda la presentación visual previa al juego usando la API de consola de Windows (SetConsoleTextAttribute, SetConsoleCursorPosition). No tiene dependencias con otros módulos del proyecto.

Función	Qué hace	Retorno
mostrarMenu()	Muestra el logo ASCII animado letra por letra, cuatro opciones navegables con W/S y una banda de fantasmas que cruzan la pantalla horizontalmente en loop.	OpcionMenu (enum)

Función	Qué hace	Retorno
mostrarMenuNivel()	Pantalla de selección de dificultad con tres opciones (FACIL / NORMAL / DIFICIL) con colores diferenciados.	int (1, 2 o 3)

6. Estrategias de resolución propuestas

6.1 Representación del mapa

Se optó por una **matriz de enteros estática** de 19×19 por su acceso $O(1)$ y simplicidad de inicialización. El patrón Original/Editable (mapaOriginal const + mapa mutable con memcpy) resuelve el problema de reinicio de partida sin necesidad de reconstruir ninguna estructura.

6.2 Pathfinding con BFS sobre grafo de adyacencia

Se construye un **grafo dirigido con listas de adyacencia** al inicio de cada partida. El BFS garantiza el camino más corto en grafos no ponderados, lo cual es correcto para una grilla donde todos los movimientos tienen el mismo costo. La reconstrucción usa un arreglo padre[] de tamaño MAX_NODOS: al encontrar el destino, se retrocede hasta el nodo cuyo padre es el origen, obteniendo el **primer paso** del camino óptimo.

6.3 Cola dinámica como estructura lineal del BFS

La cola usa **lista enlazada con nodos asignados dinámicamente** (malloc) en lugar de un arreglo estático, eliminando la necesidad de definir un tamaño máximo y cumpliendo con el requisito de la materia. Cada BFS crea y libera su propia cola (liberarCola) para evitar fugas de memoria.

6.4 Personalidades diferenciadas de fantasmas

En lugar de un único comportamiento, cada fantasma calcula un **destino objetivo distinto** y luego delega el movimiento al mismo BFS. Esto separa la estrategia (qué celda perseguir) de la táctica (cómo llegar). El campo blinkyFila/Col en el struct Grafo permite a Inky acceder a la posición de Blinky sin acoplar los structs de fantasmas entre sí.

6.5 Sistema de niveles con struct ConfigNivel

Los tres niveles se representan como un **arreglo estático de structs ConfigNivel**. Esto evita condicionales dispersos en el código: el loop del juego simplemente lee cfg.sleepMs y cfg.velocidadFantasmas sin saber en qué nivel está. El multiplicador de bonus incentiva elegir mayor dificultad.

6.6 Detección de colisiones

Se detectan dos tipos de colisión por frame: **solapamiento directo** (misma celda) y **cruce** (Pac-Man y un fantasma intercambian posiciones en el mismo turno). El segundo caso requiere guardar las posiciones anteriores de ambas entidades y verificar la condición cruzada explícitamente.

7. Cronograma tentativo de trabajo

Semana	Actividad	Módulos involucrados
1	Diseño del mapa, estructura de la grilla, renderizado básico en consola.	mapa
2	Implementación de Pac-Man: movimiento, colisión con paredes, consumo de puntos.	personajes, mapa
3	Estructura cola enlazada y grafo de adyacencia. BFS básico.	cola, grafo
4	Movimiento de fantasmas con BFS. Colisiones y vidas.	grafo, personajes, main
5	Personalidades de los cuatro fantasmas. Power-ups y modo asustado.	grafo, personajes, main
6	Menú principal animado. Selección de nivel. Tres dificultades.	menu, main
7	Sistema de puntaje, bonus por tiempo, highscores en archivo.	main
8	Pausa, reinicio de mapa, corrección de bugs. Pruebas finales.	todos
9	Redacción del informe, ajustes finales, entrega.	—

8. Herramientas y recursos utilizados

Herramienta / Recurso	Uso en el proyecto
Lenguaje C (C11)	Lenguaje de implementación principal.
Compilador g++ (MinGW)	Compilación en Windows. Flag -lwinmm para la API multimedia.
Visual Studio Code	Editor de código con extensión Code Runner para compilación y ejecución integrada.
Windows Console API	Control de cursor (SetConsoleCursorPosition), colores (SetConsoleTextAttribute), entrada sin buffer (_kbhit, _getch).
windows.h / conio.h	Headers de Windows para la API de consola y entrada de teclado.
time.h	Cronómetro de partida (time(), time_t) para el cálculo del bonus de tiempo.
stdlib.h / string.h	malloc/free para la cola dinámica; memcpy para reinicio del mapa.
Unicode (UTF-8)	Emojis como sprites de los fantasmas y caracteres de caja para la interfaz.
highscore.txt	Archivo de texto plano para persistencia de puntajes entre sesiones.

9. Conclusión

El desarrollo de este proyecto permitió integrar de forma práctica los principales conceptos de la materia: la **cola dinámica** implementada desde cero resultó fundamental para el funcionamiento del BFS, y el **grafo de adyacencia** demostró ser la estructura adecuada para modelar la navegación en una grilla con obstáculos.

Una de las decisiones de diseño más valiosas fue la separación entre la estrategia de cada fantasma (qué objetivo perseguir) y el algoritmo de movimiento (BFS), lo que permitió implementar cuatro comportamientos distintos reutilizando el mismo código de pathfinding. Esta separación de responsabilidades facilitó la depuración y la extensión del sistema.

El diseño modular en archivos separados también fue clave: al tener cada módulo con una responsabilidad clara, fue posible modificar el sistema de niveles, el menú o el reinicio del mapa sin afectar el resto del código.

Oportunidades de mejora

- Implementar guardado y carga del estado de la partida en curso (serialización del mapa y posiciones).
- Ordenar el archivo highscore.txt por puntaje de mayor a menor al guardar.
- Agregar un modo de juego con vidas infinitas o cronómetro descendente.
- Implementar múltiples niveles de mapa con diseños diferentes.
- Añadir sonidos usando la API PlaySound de winmm.
- Migrar la interfaz a una librería como ncurses o SDL para mayor portabilidad.

10. Bibliografía

- Kernighan, B. W. & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. Capítulo 22: BFS.
- Microsoft Docs. (2024). Console API Reference. <https://learn.microsoft.com/en-us/windows/console/>
- Pittman, J. (2009). The Pac-Man Dossier. <https://www.gamedeveloper.com/design/the-pac-man-dossier>
- cppreference.com — Referencia del estándar C11. <https://en.cppreference.com/w/c>
- Sedgewick, R. & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley. Capítulo de grafos y BFS.